

CKA: Logging & Monitoring

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Logging and Monitoring Section Introduction

This article covers logging and monitoring techniques for Kubernetes clusters and applications, focusing on performance tracking and log management.

Hello and welcome to this comprehensive lesson on logging and monitoring. In this session, we will explore a variety of options to effectively monitor your Kubernetes cluster components and the applications running on them.

We start by examining key monitoring techniques for Kubernetes clusters. You will learn how to keep track of both the cluster components and the individual applications to ensure optimal performance and rapid issue detection.

Next, we delve into methods for accessing and managing logs from your cluster components as well as the applications. These logs are crucial for troubleshooting issues and understanding system behavior.

Throughout this lesson, you'll engage with practical exercises designed to reinforce these concepts, ensuring you can apply what you learn in real-world scenarios.

Practice regularly using these exercises to become proficient in managing logs and monitoring your Kubernetes environment.

Let's get started on our journey to mastering logging and monitoring!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Managing Application Logs

This article provides a guide on managing application logs in Kubernetes, covering logging mechanisms in Docker and Kubernetes for effective monitoring and troubleshooting.

Welcome to this comprehensive guide on managing application logs in Kubernetes. In this article, we explore various logging mechanisms, starting with Docker and moving on to Kubernetes, to help you monitor and troubleshoot your applications effectively.

Logging in Docker

Docker containers typically log events to the standard output. Consider the "event simulator" container, which generates random events simulating a web server. When you run this container, it writes log entries such as:

```
```bash theme={null} docker run kodekloud/event-simulator 2018-10-06 15:57:15,937 - root - INFO - USER1
logged in 2018-10-06 15:57:16,943 - root - INFO - USER2 logged out 2018-10-06 15:57:17,944 - root - INFO -
USER3 is viewing page3 2018-10-06 15:57:18,951 - root - INFO - USER4 is viewing page1 2018-10-06
15:57:19,954 - root - INFO - USER1 logged out 2018-10-06 15:57:21,956 - root - INFO - USER1 logged in
2018-10-06 15:57:22,957 - root - INFO - USER3 is viewing page2 2018-10-06 15:57:23,959 - root - INFO -
USER1 logged out 2018-10-06 15:57:24,959 - root - INFO - USER2 is viewing page2 2018-10-06 15:57:25,962 -
root - INFO - USER4 is viewing page3 2018-10-06 15:57:26,965 - root - INFO - USER3 is viewing page1
2018-10-06 15:57:27,965 - root - INFO - USER3 logged out 2018-10-06 15:57:29,967 - root - INFO - USER1 is
viewing page2
```

If you run the container in detached mode using the `-d` flag, the logs will not appear on your terminal

```
```bash theme={null}
docker run -d kodekloud/event-simulator
docker logs -f <container_id>
```

Logging in Kubernetes

Deploying the same Docker image within a Kubernetes pod leverages Kubernetes' logging capabilities. To get started, create a pod using the following YAML definition:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: event-simulator-pod spec: containers: - name: event-simulator image: kodekloud/event-simulator
```

Create the pod with this command:

```
```bash theme={null}
kubectl create -f event-simulator.yaml
```

Once the pod is running, view the live logs using:

```
```bash theme={null} kubectl logs -f event-simulator-pod
```

This command outputs logs similar to the Docker example:

```
```bash theme={null}
2018-10-06 15:57:15,937 - root - INFO - USER1 logged in
2018-10-06 15:57:16,943 - root - INFO - USER2 logged out
2018-10-06 15:57:17,944 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:18,951 - root - INFO - USER3 is viewing page3
2018-10-06 15:57:20,095 - root - INFO - USER4 is viewing page1
2018-10-06 15:57:21,956 - root - INFO - USER2 logged out
2018-10-06 15:57:21,956 - root - INFO - USER1 logged in
2018-10-06 15:57:23,093 - root - INFO - USER3 is viewing page2
2018-10-06 15:57:24,959 - root - INFO - USER1 logged out
2018-10-06 15:57:25,961 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:25,961 - root - INFO - USER1 logged in
```

For more effective troubleshooting, use log filtering and analysis tools in combination with Kubernetes logs.

Logging with Multiple Containers in a Pod

Kubernetes supports pods with multiple containers. If you update your pod definition to include an additional container named `image-processor`, the configuration will look like this:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: event-simulator-pod spec: containers: - name: event-simulator image: kodekloud/event-simulator - name: image-processor image: some-image-processor
```

Create the pod using:

```
```bash theme={null}
kubectl create -f event-simulator.yaml
```

Attempting to view logs without specifying the container when multiple containers are present will result in an error. Instead, specify the container name to view its logs:

```
```bash theme={null} kubectl logs -f event-simulator-pod event-simulator
```

This command displays log output for the `event-simulator` container:

```
```bash theme={null}
2018-10-06 15:57:15,937 - root - INFO - USER1 logged in
2018-10-06 15:57:16,943 - root - INFO - USER2 logged out
2018-10-06 15:57:17,944 - root - INFO - USER2 is viewing page2
2018-10-06 15:57:18,951 - root - INFO - USER3 is viewing page3
2018-10-06 15:57:19,954 - root - INFO - USER4 is viewing page1
2018-10-06 15:57:20,955 - root - INFO - USER2 logged out
```

When working with pods that contain multiple containers, always specify the container name in the `kubectl logs` command to avoid errors.

Additional Resources

For further information on logging and monitoring in Kubernetes, refer to the following resources:

- [Kubernetes Logging Guide](#)
- [Docker Logging Drivers](#)
- [Kubernetes Documentation](#)

By mastering these logging techniques, you ensure efficient monitoring and troubleshooting of your applications in both Docker and Kubernetes environments. Happy logging!

Thank you.